BASE DE DATOS PARA EL DESARROLLO

BDDS-A06. Programación de bases de datos - I

APRIL 10, 2026

UV
Jose Eduardo Prior Hernandez

```java
package com.tienda.dto;

public class ArticuloDTO {
    private int idarticulo;
    private String nombre;
    private String descripcion;
    private int existencias;
    private double precio;

    public int getIdarticulo() {
        return idarticulo;
    }

    public void setIdarticulo(int idarticulo) {
        this.idarticulo = idarticulo;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    public int getExistencias() {
        return existencias;
    }
```

La imagen representa la clase DTO de la entidad Articulo, este cuenta con los atributos inicializados. Se inicializan de forma private para evitar que algún método los modifique directamente mente pues el diseño y patrón requiere que se la modificación y lectura de los atributos de un DTO sean a través de setters y getters.

```java
package com.tienda.DAO;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionBD {
    private static final String URL = "jdbc:mysql://localhost:3306/amazon_xalapa";
    private static final String USER = "tiendaamazonusr";
    private static final String PASS = "t13nd4AMAZON2025";

    public static Connection getConnection() {
        Connection conn = null;
        try {
            Class.forName(className: "com.mysql.cj.jdbc.Driver");
            conn = DriverManager.getConnection(URL, USER, PASS);
        } catch (ClassNotFoundException | SQLException e) {
            System.out.println("Error de conexión: " + e.getMessage());
        }
        return conn;
    }
}
```

Esta clase se ocupa de realizar la conexión de la base de datos y únicamente de la conexión de la base de datos divido al principio de cohesión y baja dependencia. Una clase solo se debe encargar 1 una cosa y tener las dependencias más bajas. A su vez bajo el principio de capas se tiene que dividir la lógica del programa en capas, en este caso se decidió dividir el programa en la capa gráfica, lógica del negocio y base de datos.

```java
package com.tienda.DAO;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import com.tienda.dto.ArticuloDTO;

public class ArticuloDAO {
    private final static String sqlInsert = "INSERT INTO articulo (nombre, descripcion, existencias, precio) VALUES (?, ?, ?, ?)";
    private final static String sqlUpdate = "UPDATE articulo SET nombre=?, descripcion=?, existencias=?, precio=? WHERE idarticulo=
    private final static String sqlDelete = "DELETE FROM articulo WHERE idarticulo = ?";
    private final static String sqlSelect = "SELECT * FROM articulo WHERE idarticulo = ?";
    private final static String sqlSelectAll = "SELECT * FROM articulo";

    public boolean agregar(ArticuloDTO articulo) {

        try (Connection con = ConexionBD.getConnection();
                PreparedStatement ps = con.prepareStatement(sqlInsert)) {
            ps.setString(parameterIndex: 1, articulo.getNombre());
            ps.setString(parameterIndex: 2, articulo.getDescripcion());
            ps.setInt(parameterIndex: 3, articulo.getExistencias());
            ps.setDouble(parameterIndex: 4, articulo.getPrecio());
            return ps.executeUpdate() > 0;
        } catch (SQLException e) {
            System.err.println("Error al agregar: " + e.getMessage());
            return false;
        }
    }
```

Se inicializan los stament para realizar las query deseadas en la base de datos, se ocupan final static pues no se van a modificar las quieries, al cargar la query es cuando se cambian los ? por los valores deseados en la query. Tambien se crea el método para insertar artículos a la base de datos, note que no se pone el índice del articulo pues es incremental y no se requiere de agregarlo.

```java
public boolean actualizar(ArticuloDTO articulo) {

    try (Connection con = ConexionBD.getConnection();
            PreparedStatement ps = con.prepareStatement(sqlUpdate)) {
        ps.setString(parameterIndex: 1, articulo.getNombre());
        ps.setString(parameterIndex: 2, articulo.getDescripcion());
        ps.setInt(parameterIndex: 3, articulo.getExistencias());
        ps.setDouble(parameterIndex: 4, articulo.getPrecio());
        ps.setInt(parameterIndex: 5, articulo.getIdarticulo());
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error al actualizar: " + e.getMessage());
        return false;
    }
}

public boolean eliminar(int idarticulo) {

    try (Connection con = ConexionBD.getConnection();
            PreparedStatement ps = con.prepareStatement(sqlDelete)) {
        ps.setInt(parameterIndex: 1, idarticulo);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error al eliminar: " + e.getMessage());
        return false;
    }
}
```

```java
public ArticuloDTO buscar(int idarticulo) {

    ArticuloDTO articulo = null;
    try (Connection con = ConexionBD.getConnection();
            PreparedStatement ps = con.prepareStatement(sqlSelect)) {
        ps.setInt(parameterIndex: 1, idarticulo);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
            articulo = new ArticuloDTO();
            articulo.setIdarticulo(rs.getInt(columnLabel: "idarticulo"));
            articulo.setNombre(rs.getString(columnLabel: "nombre"));
            articulo.setDescripcion(rs.getString(columnLabel: "descripcion"));
            articulo.setExistencias(rs.getInt(columnLabel: "existencias"));
            articulo.setPrecio(rs.getDouble(columnLabel: "precio"));
        }
    } catch (SQLException e) {
        System.err.println("Error al buscar: " + e.getMessage());
    }
    return articulo;
}

public List<ArticuloDTO> listarCompleto() {

    List<ArticuloDTO> lista = new ArrayList<>();
    try (Connection con = ConexionBD.getConnection();
            PreparedStatement ps = con.prepareStatement(sqlSelectAll);
            ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            ArticuloDTO articulo = new ArticuloDTO();
            articulo.setIdarticulo(rs.getInt(columnLabel: "idarticulo"));
            articulo.setNombre(rs.getString(columnLabel: "nombre"));
            articulo.setDescripcion(rs.getString(columnLabel: "descripcion"));
            articulo.setExistencias(rs.getInt(columnLabel: "existencias"));
            articulo.setPrecio(rs.getDouble(columnLabel: "precio"));
            lista.add(articulo);
```

Se crean los demás métodos, utilizando los statement adecuados para la query deseada, el preparedStatemnt es una palabra reservada que se utiliza para decir que el string es una query a insertar en una base de datos de Mysql, y se prepara para insertar la operación en el sistema gestor de la base de datos. También impide que se inserten código en la string, protegiendo contra ataques maliciosos.

```java
package com.tienda.vista;

import javax.swing.*;
import java.awt.*;
import com.tienda.dto.ArticuloDTO;
import com.tienda.DAO.ArticuloDAO;

public class RegistrarArticuloForm extends JInternalFrame {
    // Definición de componentes
    private JTextField txtNombre, txtDesc, txtExistencia, txtPrecio;
    private JButton btnAgregar, btnSalir;

    public RegistrarArticuloForm() {
        setTitle(title: "Registrar Nuevo Artículo");
        setClosable(b: true);
        setSize(width: 400, height: 300);
        setLayout(new GridLayout(rows: 6, cols: 2, hgap: 10, vgap: 10));

        add(new JLabel(text: " Nombre:"));
        txtNombre = new JTextField();
        add(txtNombre);

        add(new JLabel(text: " Descripción:"));
        txtDesc = new JTextField();
        add(txtDesc);

        add(new JLabel(text: " Existencias:"));
        txtExistencia = new JTextField();
        add(txtExistencia);

        add(new JLabel(text: " Precio:"));
        txtPrecio = new JTextField();
        add(txtPrecio);

        btnAgregar = new JButton(text: "Agregar");
        btnSalir = new JButton(text: "Salir");
```

```java
private void accionAgregar() {
    try {
        ArticuloDTO dto = new ArticuloDTO();
        ArticuloDAO dao = new ArticuloDAO();

        dto.setNombre(txtNombre.getText());
        dto.setDescripcion(txtDesc.getText());
        dto.setExistencias(Integer.parseInt(txtExistencia.getText()));
        dto.setPrecio(Double.parseDouble(txtPrecio.getText()));

        if (dao.agregar(dto)) {
            JOptionPane.showMessageDialog(this, message: "¡Artículo guardado en Amazon Xalapa!");
            limpiarCampos();
        } else {
            JOptionPane.showMessageDialog(this, message: "Error al guardar.", title: "Error", JOptionPane.ERROR_MESSAGE);
        }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, message: "Por favor, revisa los campos numéricos.", title: "Aviso",
                JOptionPane.WARNING_MESSAGE);
    }
}

private void limpiarCampos() {
    txtNombre.setText(t: "");
    txtDesc.setText(t: "");
    txtExistencia.setText(t: "");
    txtPrecio.setText(t: "");
}
```

Esta clase se ocupa de crear y formatear el formulario a llenar para registrar un articulo a la base de datos, note que la clase no sabe el como se guardan los datos en la base de datos o el como se realiza la consulta, solo se encarga de llegar el dto con los datos introducidos y se los pasa al dao para que este sea quien realice e inserte los datos a la base de datos.

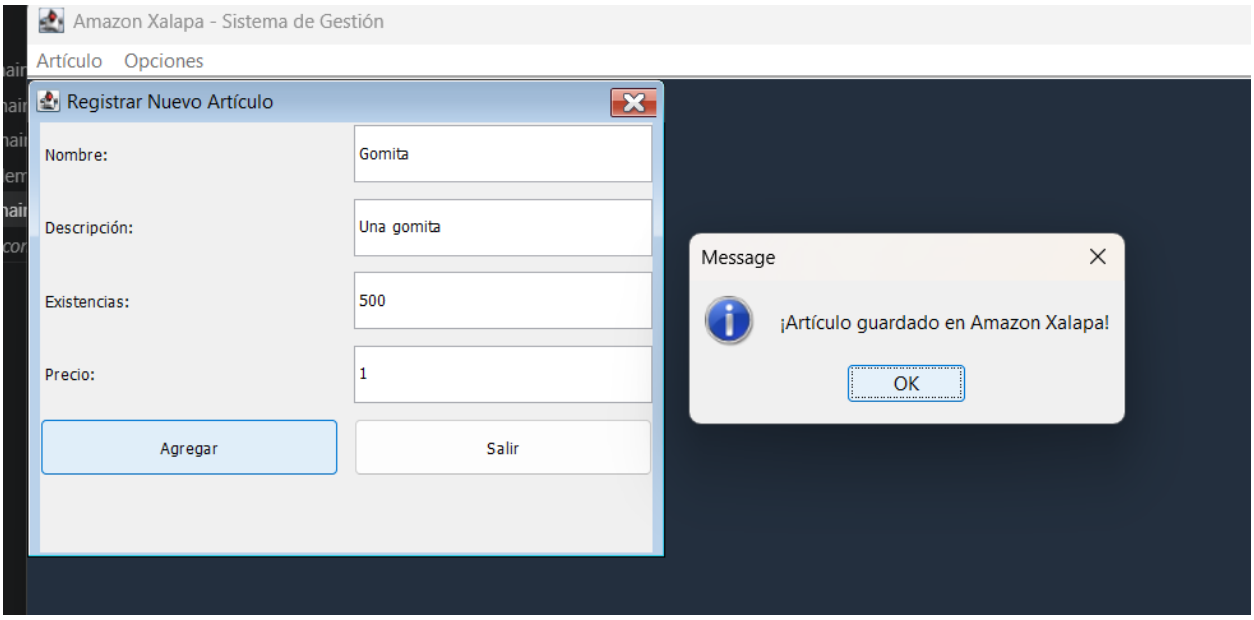
```java
package com.tienda.vista;

import javax.swing.*;
import java.awt.*;

public class FrmPrincipal extends JFrame {

    private JDesktopPane escritorio;

    public FrmPrincipal() {
        initComponents();
    }

    private void initComponents() {

        setTitle(title: "Amazon Xalapa - Sistema de Gestión");
        setSize(width: 1000, height: 700);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(c: null);

        escritorio = new JDesktopPane();
        escritorio.setBackground(new Color(r: 35, g: 47, b: 62));
        add(escritorio, BorderLayout.CENTER);

        JMenuBar menuBar = new JMenuBar();

        JMenu menuArticulo = new JMenu(s: "Artículo");

        JMenuItem itemRegistrar = new JMenuItem(text: "Registrar Artículo");
        JMenuItem itemModificar = new JMenuItem(text: "Modificar Artículo");
        JMenuItem itemEliminar = new JMenuItem(text: "Eliminar Artículo");

        JMenu subMenuConsultar = new JMenu(s: "Consultar");
        JMenuItem itemBuscarID = new JMenuItem(text: "Buscar por ID");
        JMenuItem itemBuscarTodos = new JMenuItem(text: "Buscar todos");

        subMenuConsultar.add(itemBuscarID);
```

```java
JMenu menuSalir = new JMenu(s: "Opciones");
JMenuItem itemCerrar = new JMenuItem(text: "Salir del Sistema");
menuSalir.add(itemCerrar);

menuBar.add(menuArticulo);
menuBar.add(menuSalir);
setJMenuBar(menuBar);

itemRegistrar.addActionListener(e -> {
    RegistrarArticuloForm registro = new RegistrarArticuloForm();
    escritorio.add(registro);
    registro.setVisible(aFlag: true);
});

itemBuscarID.addActionListener(e -> {
});

itemBuscarTodos.addActionListener(e -> {
});

itemCerrar.addActionListener(e -> System.exit(status: 0));
```

```java
Run | Debug
public static void main(String[] args) {
    try {
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
    } catch (Exception e) {
        e.printStackTrace();
    }

    SwingUtilities.invokeLater(() -> {
        new FrmPrincipal().setVisible(b: true);
    });
}
```

Esta clase es la clase principal para la lógica de las Gui, sirve como ventana principal y es donde se cargarán las demás ventas. En este caso se utiliza para abrir, e inicializar el formulario para ingresar los datos de los artículos.

De la misma forma se utilizara para cargar las interfaces utilizadas para visualizar el resultado de las consultas, y realizar otras operaciones pertinentes




```
mysql> select * from articulo;
+------------+--------+-------------+-------------+--------+
| idarticulo | nombre | Descripcion | existencias | precio |
+------------+--------+-------------+-------------+--------+
|          1 | Paleta | una paleta  |           2 |     10 |
|          2 | Gomita | Una gomita  |         500 |      1 |
+------------+--------+-------------+-------------+--------+
2 rows in set (0.008 sec)
```

¿Qué aprendí en esta práctica?

Aprendí el cómo se conectan las bases de datos en el lenguaje de programación java, así del como se realizan las ventanas con las que interactuamos cada día, el cómo se actualizan/cambian elementos dentro de estas ventanas. A su vez, también aprendí un sobre estándares de programación, de cómo se podría dividir un proyecto para mantener la lógica precisa y limpia, el como se ocultan ciertas partes de programa de otros subsistemas, el como se crea la modularidad y buenas practicas a la hora de organizar el proyecto a trabajar.